Solution: Best Meeting Point

Problem Statement

You are given `m x n` matrix containing `1` and `0` digits only, where each `1` represents the home of one friend, return the minimum `total travel distance`.

The `minimal total travel distance` is defined as a sum of distances between each friend's house and meeting point. Here, you need to find the meeting point such that it should have minimal distance from the home of each friend.

The distance is calculated as the Manhattan distance, which is the sum of the absolute differences in the horizontal and vertical coordinates `(distance(a1, a2) = |a2.x - a1.x| + |a2.y - a1.y|)`.

Example

Example 1:

- **Input**:

```
[[1,0],
 [0,1]]
```

- **Expected Output**: 2
- **Justification**: There are two people located at the top-left and bottom-right corners. The best meeting point is either of the two empty cells, each resulting in a total distance of 2.

Example 2:

- **Input**:

```
[[1,0,1],
 [0,1,0],
 [1,0,1]]
```

- **Expected Output**: 8
- **Justification**: The grid has four people at the corners and one in the center. The best meeting points are the four cells adjacent to the center cell (either horizontally or vertically), each resulting in a total distance of 8.

Example 3:

- **Input**:

```
[[1,1,0],
 [0,0,1],
 [0,0,1]]
```

- **Expected Output**: 6
- **Justification**: People are located at the top two corners and the bottom two cells of the rightmost column. The optimal meeting point is the cell in the second row, center, or rightmost column, with a total distance of 6. This point is vertically central and at the horizontal edge, closest to the majority of people.

Solution

To solve this problem, we leverage the properties of Manhattan distance. Since the distance is the sum of absolute differences along each axis, we can treat the x and y coordinates independently. The optimal meeting point minimizes the sum of horizontal distances for all x coordinates and vertical distances for all y coordinates.

The most effective approach is to find the median of all x and y coordinates of the people on the grid. The median minimizes the sum of absolute differences from a central point. First, we collect all the x and y coordinates of the people, then find their medians. The cell at the median x and median y coordinates is our optimal meeting point. This approach works because in a sorted list of numbers, the median minimizes the sum of absolute differences.

Step-by-step Algorithm

- Initialize two lists, `xCoords` and `yCoords`, to store the x and y coordinates of people.
- Traverse the grid:
  - For each cell with a person (1), add its x and y coordinates to `xCoords` and `yCoords` respectively.
- Sort both `xCoords` and `yCoords`.
- Find the median of `xCoords` and `yCoords`. If the number of coordinates is even, choose the lower middle value as the median.
- Calculate the total Manhattan distance:
  - Sum up the absolute differences between each coordinate in `xCoords` and the x median.
  - Do the same for `yCoords` and the y median.
- Return the total sum of these distances as the result.

5. Algorithm Walkthrough

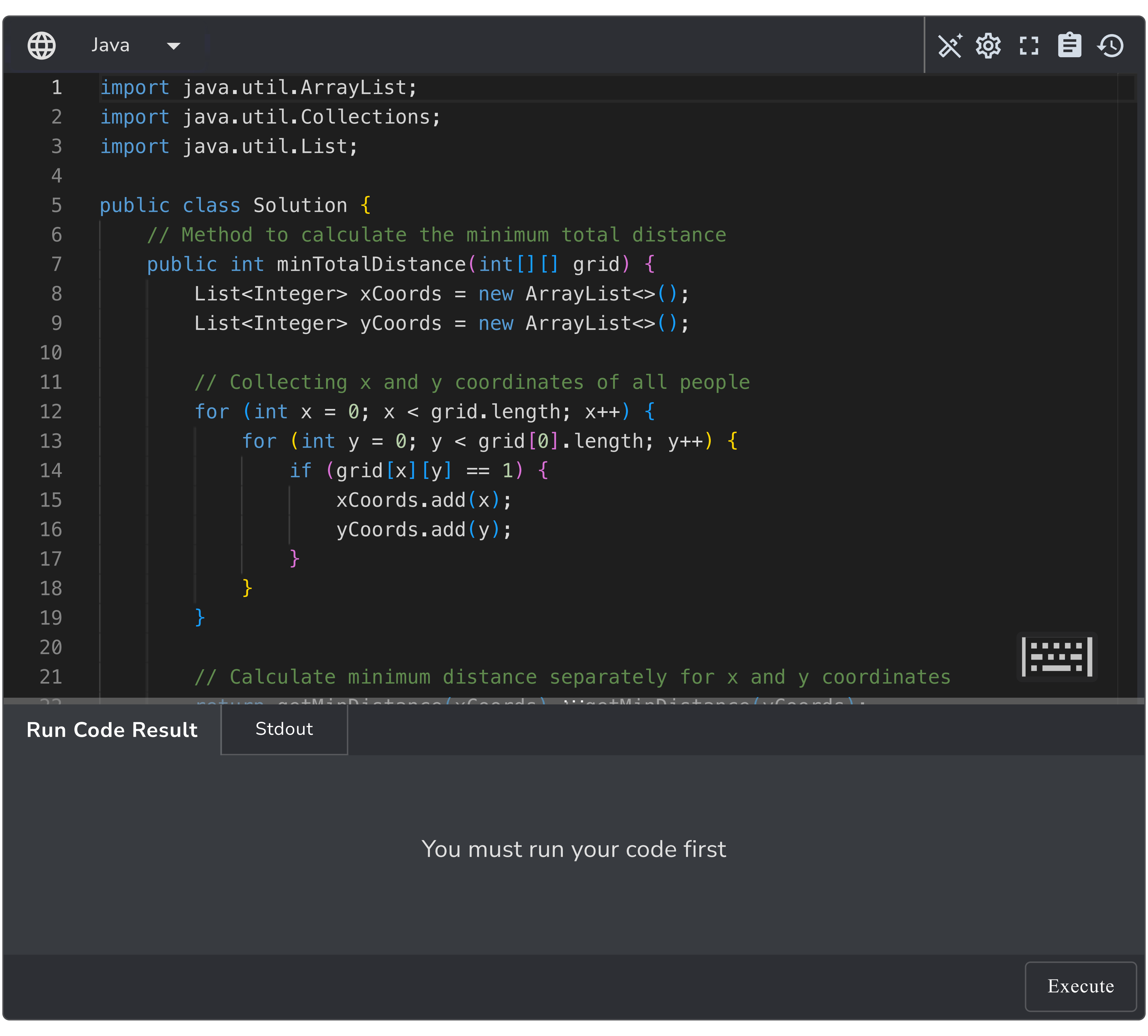
Let's take Example 2:

```
[[1, 0, 1],
 [0, 1, 0],
 [1, 0, 1]]
```

1. **Collect Coordinates**:
   - Traverse the grid and collect the x and y coordinates of all cells containing 1.
   - Resulting coordinates:
     - `xCoords = [0, 0, 1, 2, 2]`
     - `yCoords = [0, 2, 1, 0, 2]`
2. **Sort Coordinates**:
   - Sort both `xCoords` and `yCoords` independently.
   - After sorting:
     - `xCoords = [0, 0, 1, 2, 2]` (already sorted)
     - `yCoords = [0, 0, 1, 2, 2]` (after sorting)
3. **Calculate Median**:
   - For both lists, the median is the middle element (as we have an odd number of elements).
   - Median of `xCoords` = 1 (middle element)
   - Median of `yCoords` = 1 (middle element)
4. **Calculate Total Distance**:
   - Sum the absolute differences between each coordinate and the median.
   - For `xCoords`: |0-1| + |0-1| + |1-1| + |2-1| + |2-1| = 1 + 1 + 0 + 1 + 1 = 4
   - For `yCoords`: |0-1| + |0-1| + |1-1| + |2-1| + |2-1| = 1 + 1 + 0 + 1 + 1 = 4
5. **Final Result**:
   - Total distance = Distance for `xCoords` + Distance for `yCoords` = 4 + 4 = 8

Hence, for the given grid, the minimum total distance to the best meeting point is 8.

Code



```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Solution {
    // Method to calculate the minimum total distance
    public int minTotalDistance(int[][] grid) {
        List<Integer> xCoords = new ArrayList<>();
        List<Integer> yCoords = new ArrayList<>();

        // Collecting x and y coordinates of all people
        for (int x = 0; x < grid.length; x++) {
            for (int y = 0; y < grid[0].length; y++) {
                if (grid[x][y] == 1) {
                    xCoords.add(x);
                    yCoords.add(y);
                }
            }
        }

        // Calculate minimum distance separately for x and y coordinates
```

You must run your code first

Complexity Analysis

Time Complexity

$O(m * n + k * logk)$, where `m` and `n` are the dimensions of the grid, and `k` is the number of people in the grid. This is due to the initial grid traversal to collect coordinates $O(m * n)$ and the subsequent sorting of these coordinates $O(k * logk)$.

Space Complexity

$O(k)$, where `k` is the number of people in the grid. This space is used to store the coordinates of the people. In the worst-case scenario, if every cell is occupied, this becomes $O(m * n)$.

← Previous
Best Meeting Point (Hard)

Next →
Maximum Sum Of 3 Non-Overlapping Subarrays (Hard)

☐ Mark as Completed